

Code of the Day

- Apologize again for the zoom lecture as I am attending ASPLOS at Netherlands now
 - Please mute yourself and turn off the video – my hotel internet is only 5Mbps ...
 - In case the internet quality is poor, you can drop off; the lecture will be recorded (sorry about this!)
- What is the problem for the following kernel if launched via **kernel<<<1, 32>>>()** ?

```
__global__ void kernel() {  
    __shared__ int shared_data[32];  
    int tid = threadIdx.x;  
    shared_data[tid] = tid;  
    if (tid % 2 == 0) {  
        __syncthreads();  
    }  
    shared_data[tid] += 1;  
}
```

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

GPU Memory Performance Optimization

03/31/2025

Before we get started...

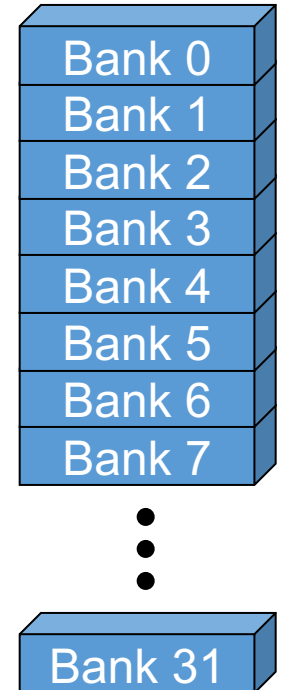
- Quick overview, things discussed last time
 - Shared memory: a block-level memory space with fast access latency by threads
 - The `__syncthreads()` function: a primitive to synchronize all threads within the same block
 - Local memory, constant memory & other aspects related to the movement of data between GPU & CPU
- Purpose of today's lecture:
 - Shared memory performance optimization
 - Global memory performance optimization
- Miscellaneous
 - Programming assignment #6 is released – due **4/14 at 23:50 PM**
 - Due to travel conflict, we will **NOT** have the lecture on 4/4 (this Friday)
 - Please also take this time to work more on your final project!

Final Project Presentation Action Items: 10% of your Grade

- Action #1: Sign up for your presentation slot [here](#)
 - Every team must sign up one 10-min slot
- Action #2: Complete your presentation slides based on the provided template
 - Each team has 10 minutes – consider this as an “elevator pitch” of your project
 - Again, keep in mind – 70% of your career success goes to your presentation/communication skills!
- Action #3: Upload your slides (in PDF) to Canvas by 12 PM (noon) of your presentation date
 - There will be three assignment pages opened for 4/25, 4/28, 5/2
 - For instance, if you sign up for 4/25, you must upload your presentation slide by 12 PM (noon) on 4/25
 - Every team member MUST submit the same presentation slide – like your proposal
- Action #4: Show up on time on your presentation day

How is Shared Memory Physically Structured & How it Works [1/2]

- Shared memory (ShMem) is accessible to all threads **within the same block**
- During block execution, many threads access the ShMem simultaneously
 - To serve more than one thread, ShMem is divided into independent **banks**
 - This layout essential to achieve high bandwidth and throughput performance
- Each SM organizes ShMem in 32 memory banks and schedule them *bank-by-bank*
 - Inter-bank access will run in parallel
 - Intra-bank access will run in sequential
- Each bank has a bandwidth of 32 bits per clock cycle
 - You can access only 32-bit or 4-byte data per clock cycle from each bank
- Successive 32-bit (4-byte) words in ShMem map to successive banks



Example: Mapping Shared Memory to Banks

- Assume `__shared__ int A[1024]`:
 - `int` is of 4 bytes (32 bits), taking one bank

A[0] → Bank 0
A[1] → Bank 1
A[2] → Bank 2
...
A[31] → Bank 31
A[32] → Bank 0 (Wraps around)
A[33] → Bank 1
A[34] → Bank 2
...

So, $A[n]$ maps to Bank $(n \% 32)$

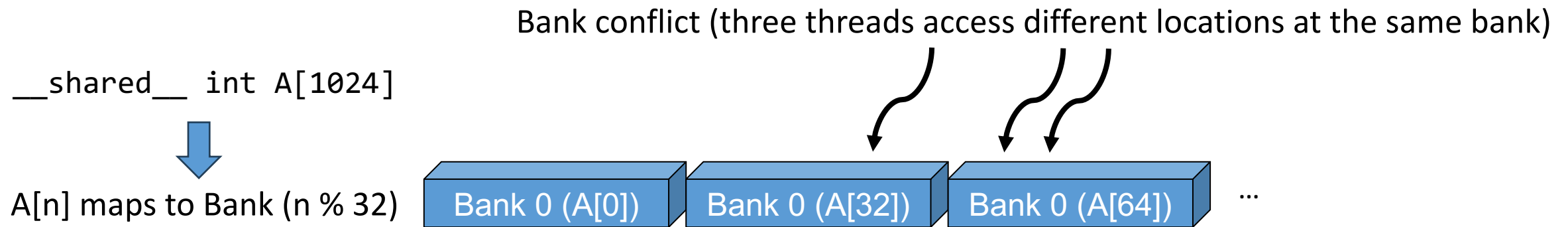
- Assume `__shared__ double B[1024]`:
 - `double` is of 8 bytes (64 bits), taking two banks

B[0] → Bank 0, Bank 1
B[1] → Bank 2, Bank 3
B[2] → Bank 4, Bank 5
...
B[15] → Bank 0, Bank 1 (Wraps around)
B[16] → Bank 2, Bank 3
B[17] → Bank 4, Bank 5
B[18] → Bank 6, Bank 7
...

So, $B[n]$ maps to Bank $(2n \% 32)$ and $((2n+1) \% 32)$

Shared Memory: Transaction Rules & Bank Conflicts

- When reading in 4-byte words, 32 threads **in a warp** attempt to access shared memory simultaneously
- **Bank conflict**: two different threads access *different* words of the **same bank**



- Bank conflicts force the hardware to serialize a ShMem access, which adversely impacts bandwidth
- IMPORTANT: “bank conflicts” occurs in the scope of one warp

Best Case and Worst Case of Shared Memory

- If there are no bank conflicts:
 - Shared memory access is fast, but not quite as fast as register access
 - On the bright side, latency is roughly 50-100x faster than global memory latency
- ShMem operation, the fast case:
 - If all threads within a warp access different banks, there is no bank conflict
 - Thread-to-bank access can be arbitrary as long as they don't overlap, e.g., thread 0 access bank 13 while thread 1 accesses bank 4, etc.
- ShMem memory operation, the slow case:
 - Worst case: 32 threads access 32 different words in the same bank – all accesses are serialized
 - In general, cost of ShMem in cycles proportional to the max # of simultaneous accesses to one bank

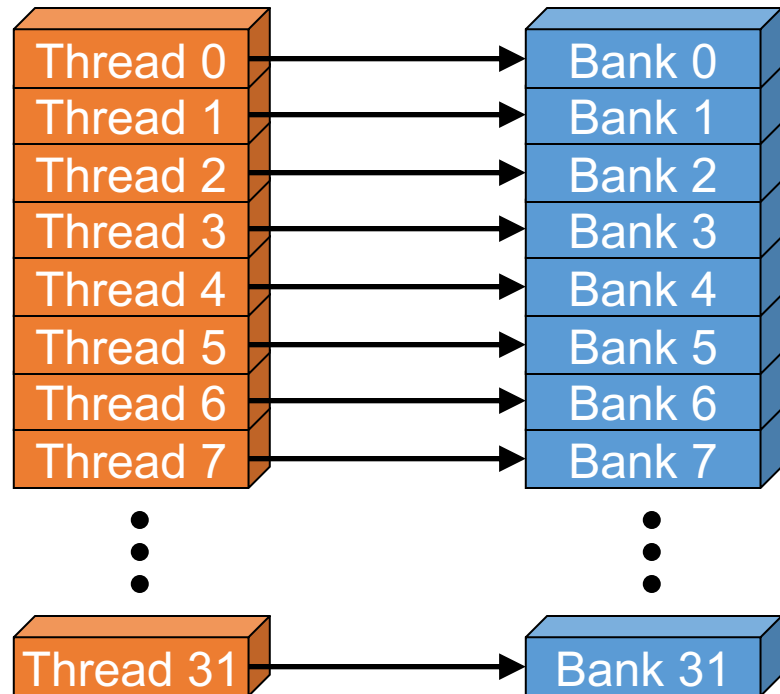
Bank Conflict Occurs Only Within a Warp

- There is no such thing as bank conflicts between threads belonging to different warps
 - Bank conflicts only occur within a single warp (32 threads)
 - CUDA executes warps independently, meaning threads from different warps access shared memory in separate transactions
 - A bank conflict happens only when multiple threads in the same warp access different addresses in the same shared memory bank simultaneously
- **Within a warp:**
 - If multiple threads access data from the same bank, the accesses get **serialized**, leading to a **bank conflict**
- **Between different warps:**
 - Each warp is scheduled independently, and shared memory accesses from different warps are handled in separate memory transactions—they don't interfere with each other.

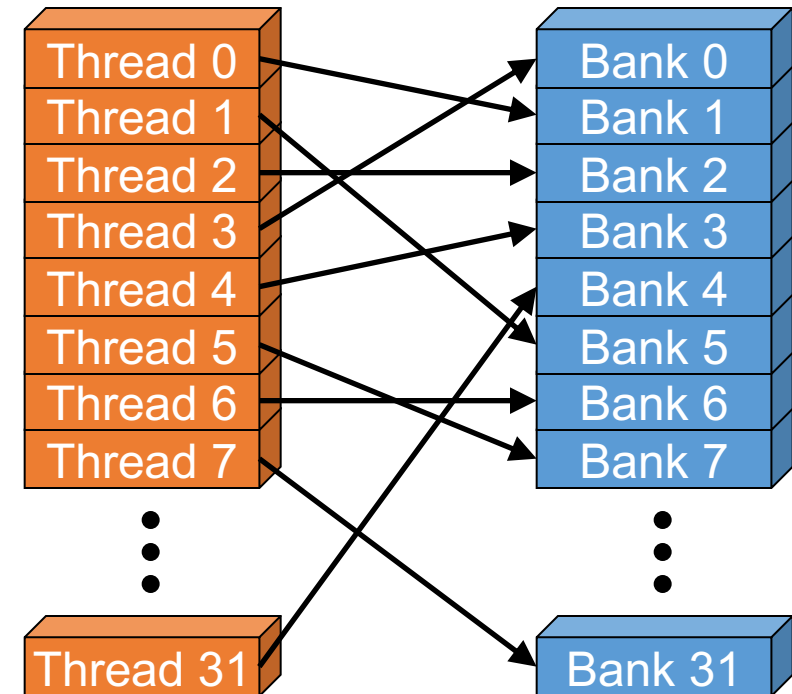
Bank Addressing Examples

Transactions Involving 4 Byte Words

- No Bank Conflicts
 - Linear addressing stride == 1



- No Bank Conflicts
 - Random 1:1 Permutation

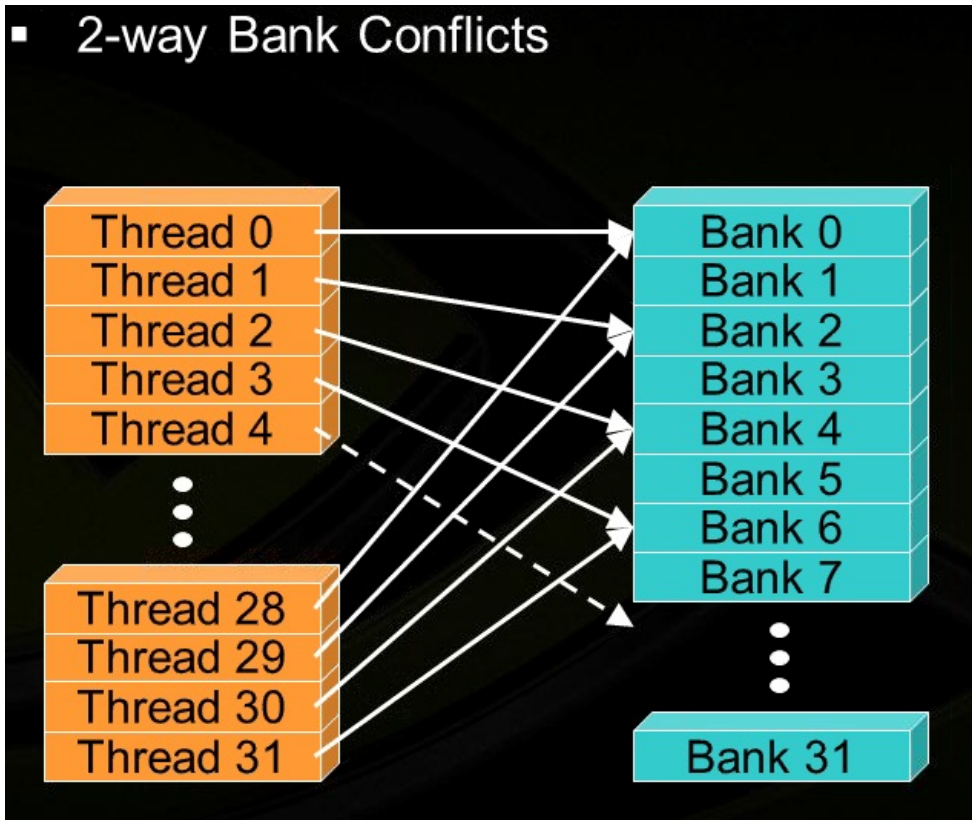


Bank Addressing Examples

Transactions Involving 4 Byte Words

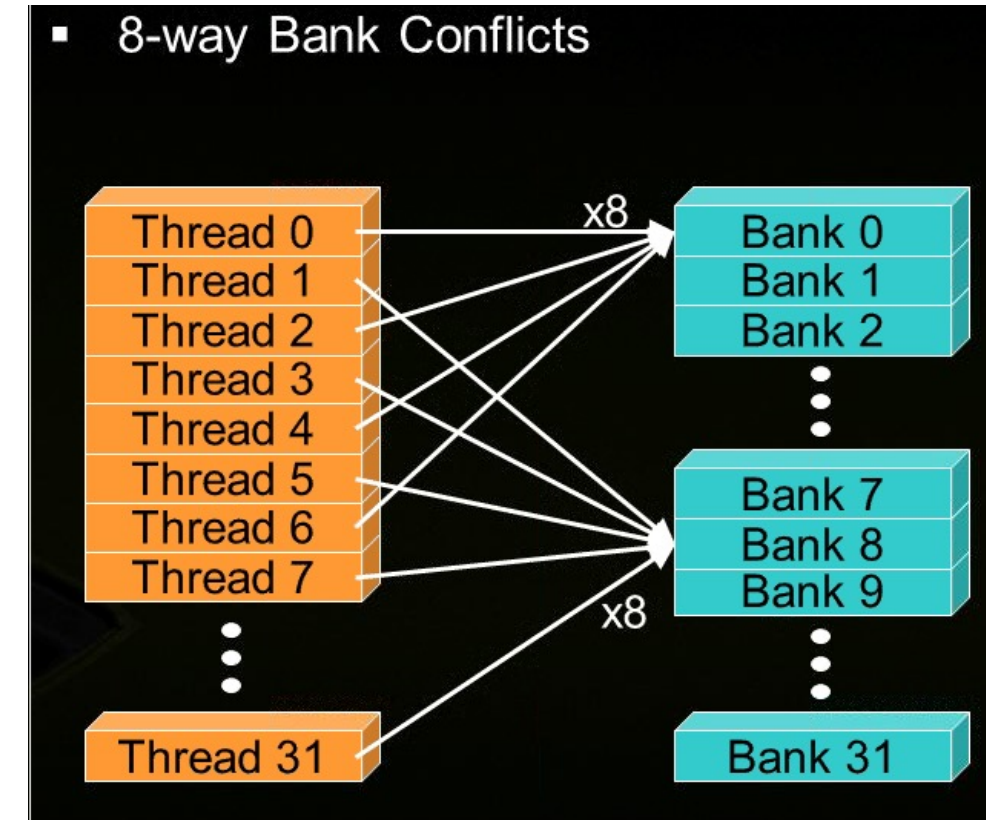
A case in which only the even banks end up being accessed, and the warps access different words in each bank

2-way Bank Conflicts



A case in which the 32 threads in a warp access only banks 0, 8, 16, and 24 (and different words in these four banks)

8-way Bank Conflicts



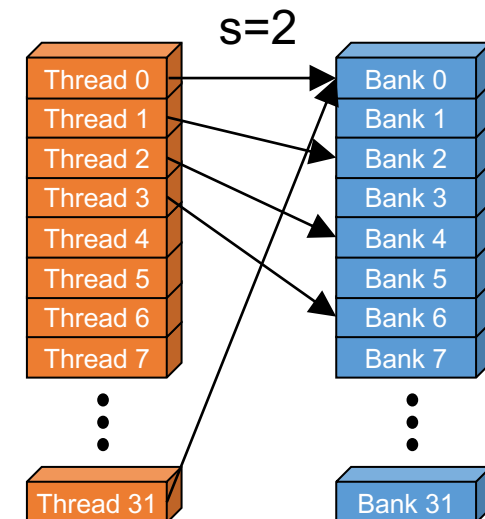
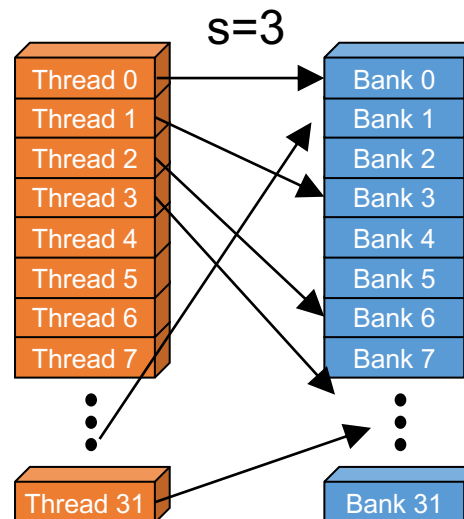
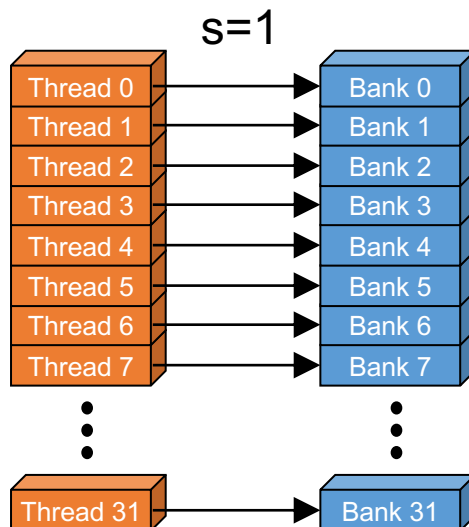
Use the Right Linear Address to Avoid Bank Conflict

- Given `__shared__ float sharedM[256];`

```
float foo = sharedM[baseIndex + s * threadIdx.x];
```

↑ (iterate in a stride size of $s * \text{threadIdx.x}$)

- This is bank-conflict-free if s shares no common factors with 32 (the number of banks)
 - Why? ($s * \text{threadIdx.x} \% 32$) will skip all odd numbers if s is even
 - Conclusion: you are fine if s is odd



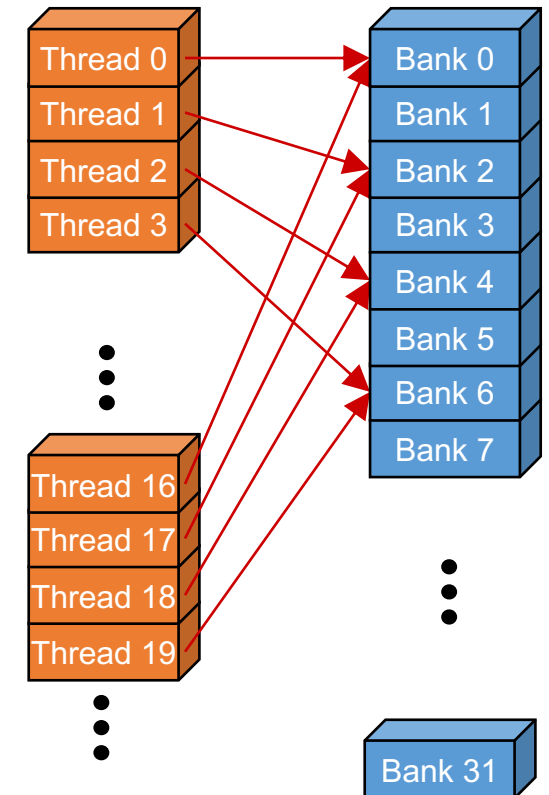
Note: baseIndex assumed 0 (zero) in these two pics

Another Bank Conflict Example

- Snippet from a kernel, each thread loads two floats into ShMem:

```
int tid = threadIdx.x;  
int offset = 128;  
shared[2*tid] = global[offset+2*tid];
```

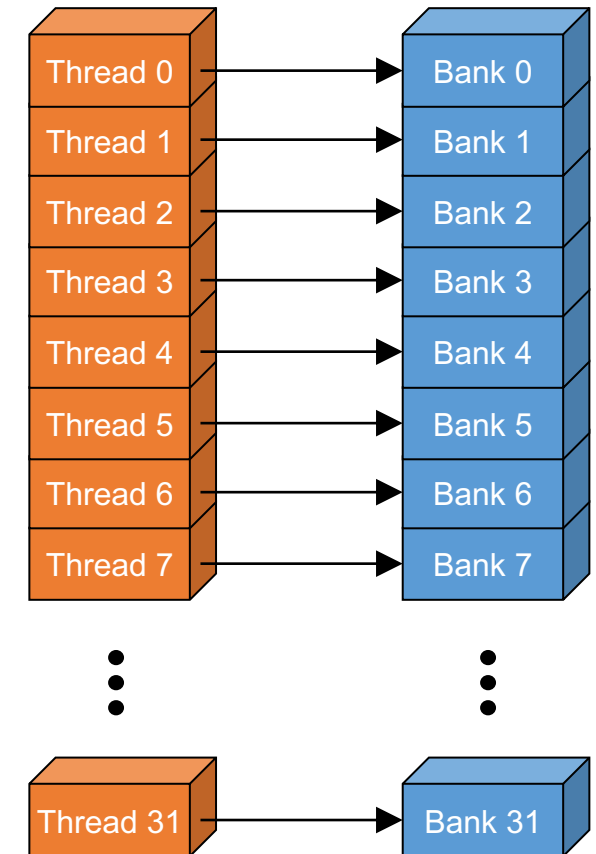
- This code makes sense for CPU threads as it prevents false sharing
- However, this code doesn't make sense in shared memory usage where there is no cache line effects but banking effects
 - 2-way-interleaved loads result in 2-way bank conflicts
- Adding insult to injury: you don't have coalesced global memory loads – basically you are halving the global mem bandwidth (discuss later)



Quiz: Revisiting example on previous slide

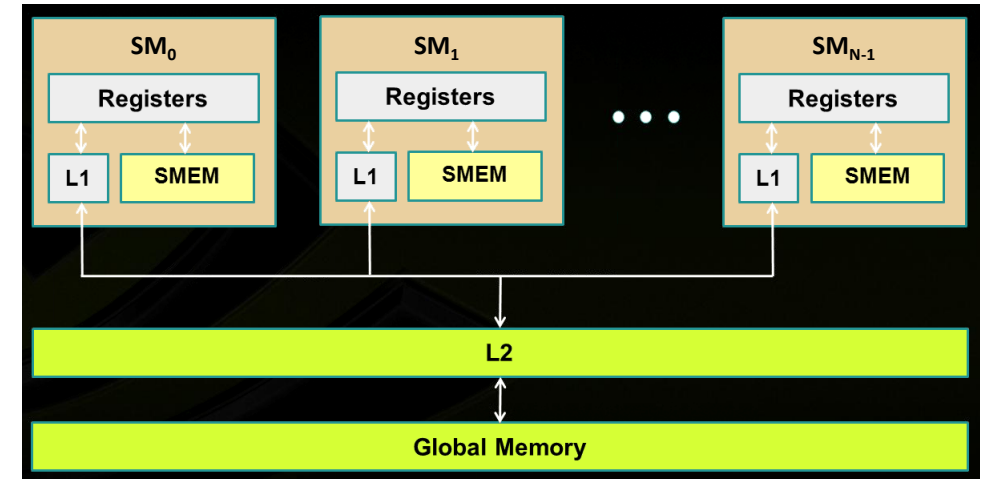
- Is this a good way of accessing ShMem?
 - Each thread loads one element in a stride size of blockDim

```
shared[tid] = global[offset + tid];  
shared[tid + blockDim.x] = global[offset + tid + blockDim.x];
```



Global Memory in GPU

- All global memory accesses are cached
 - Data must go through L2 cache
- On the GPU, a cache line is **128 bytes**
 - Mapped to a 128-byte aligned segment in global memory (PS: in CPU, the cache line is 64 bytes)
 - Why 128 bytes? $128 \text{ bytes} = 32 \text{ (warp size)} * 4 \text{ bytes}$
 - In other words, 32 floats or 32 ints fit in one cache line (and can be brought in L2 or L1 in one trip)
- If the size of the type accessed by each thread is more than 4 bytes, a memory request by a warp is first split into separate 128-byte memory requests that are issued independently



Similar to Shared Memory, Global Memory Latency is Different

- Not all **shared** memory accesses are equally efficient
 - Solution: Optimize the shared memory access to avoid bank conflicts
- Not all **global** memory accesses are equally efficient
 - How can you optimize memory access?
- Getting global memory access right has **higher** priority
 - Why? Global memory latency is much higher (slower) than shared memory

Global Memory Access Should Stay Away from “Divergence”

- The concept of “divergence” is similar to thread divergence, but a bit different in the sense that thread divergence deals with instructions and global memory deals with memory access
- Hardware is optimized for accessing contiguous blocks of global memory when performing loads and stores
- “block of global memory”: a 128-byte aligned chunk of 128 byte of memory
 - Becomes a GPU cache line, once it gets read in L2 (and possibly L1) memory

Global Memory Access are Pertain to a Warp of Threads

- There are two important attributes in global memory access when fetching data from global memory into shared memory and/or registers
 - Attribute 1: The **layout of the access** to global memory (the pattern of the access)
 - Attribute 2: The **alignment** of the data you try to fetch from global memory
- IMPORTANT:
 - The two attributes in red should be interpreted in the context of a **global memory transaction** carried out by **one warp** of threads
 - That is, not by one thread, not all threads in a block, not all threads in a grid, etc. But one warp of threads

Attribute #1: Layout of the Access

- The basic idea:
 - Suppose each thread in a warp accesses a global memory address for a load operation at some point in the execution of the kernel
 - These threads can access global memory data that is either (a) neatly grouped, or (b) scattered all over the place
 - Case (a) is called a “coalesced memory access” – basically spatial locality
 - If you end up with (b) this will adversely impact the effective bandwidth of your program

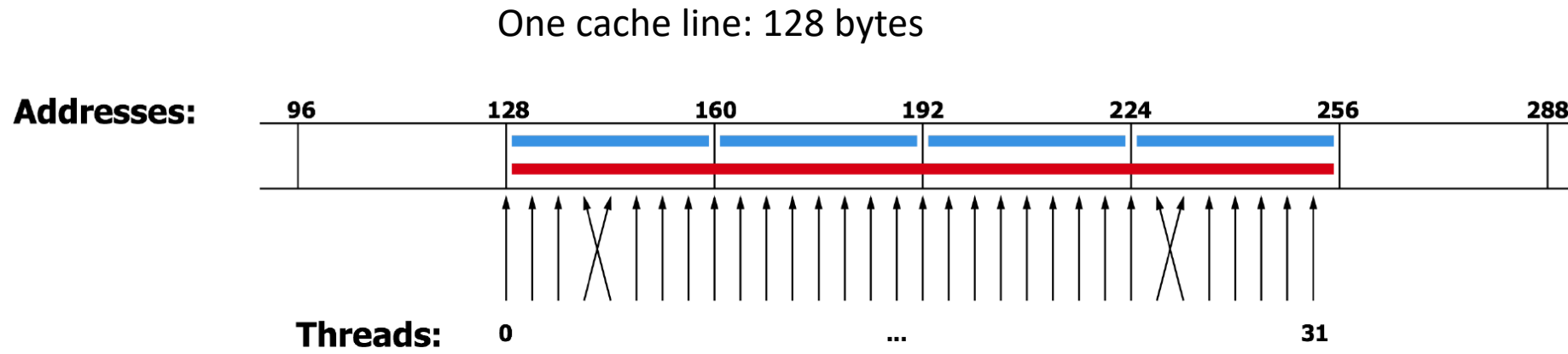
Attribute #2: Alignment

- Fact: Any address of device memory allocated in with `cudaMalloc` is a multiple of 256
- The “alignment” component of this story:
 - It’s great when all threads in a warp access data inside only one memory block

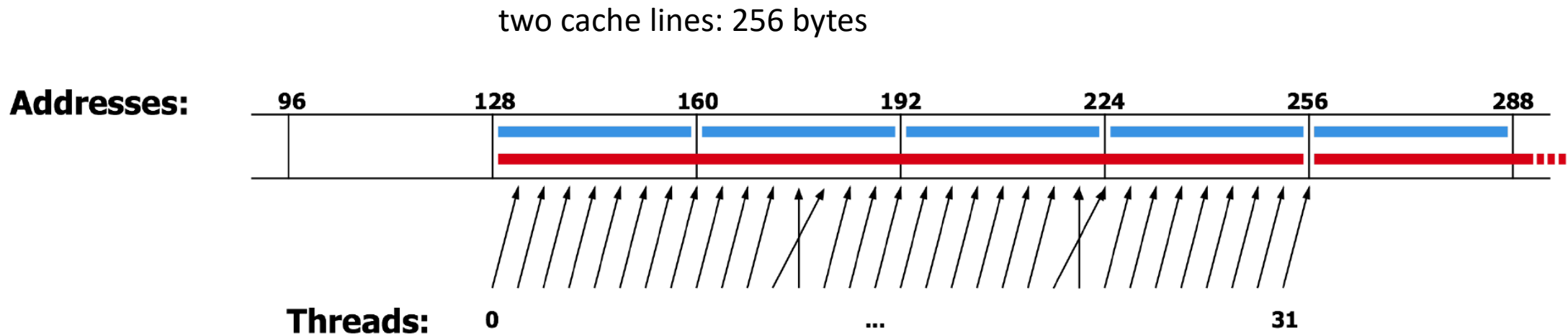
Best Global Memory Access Case

- Best memory accesses are simultaneously: (1) coalesced and (2) properly aligned

Scenario A: Coalesced and aligned – great!



Scenario B: Coalesced, but not aligned – still okay though



Example: Adding Two Matrices

- You have two matrices A and B of dimension NxN (N=32)
- You want to compute $C=A+B$ in parallel

Beginning of content of file add2Mat.cu

```
#include <iostream>
#include <cuda_runtime.h>

const int N = 32;

// Kernel definition
__global__ void MatAdd(float A[N][N], float B[N][N], float C[N][N]) {
    int i = threadIdx.x;
    int j = threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}
```

Content of file add2Mat.cu continues here

```
int main() {
    float A[N][N], B[N][N], C[N][N];
    float (*d_A)[N], (*d_B)[N], (*d_C)[N];

    // Initialize matrices A and B with sample values
    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++) {
            A[i][j] = i + j;
            B[i][j] = i - j;
        }

    // Allocate device memory
    cudaMalloc((void**)&d_A, N*N*sizeof(float));
    cudaMalloc((void**)&d_B, N*N*sizeof(float));
    cudaMalloc((void**)&d_C, N*N*sizeof(float));

    // Copy data to device
    cudaMemcpy(d_A, A, N*N*sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, N*N*sizeof(float), cudaMemcpyHostToDevice);

    // Kernel invocation
    dim3 threadsPerBlock(N, N);
    MatAdd<<<1, threadsPerBlock>>>(d_A, d_B, d_C);

    // Copy results back to host
    cudaMemcpy(C, d_C, N*N*sizeof(float), cudaMemcpyDeviceToHost);

    // Clean up
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}
```


Test Your Understanding

- Given that the x field of a thread changes the fastest, is the array indexing scheme on MatAdd good or bad?
 - HINT: when forming a warp, think how threads are mapped into a set of 1D sequence of consecutive IDs
- The “good or bad” refers to how data is accessed in the device’s global memory

- In other words, should we have

`C[i][j] = A[i][j] + B[i][j]`

or...

`C[j][i] = A[j][i] + B[j][i]`

```
#include <iostream>
#include <cuda_runtime.h>

const int N = 32;

// Kernel definition
__global__ void MatAdd(float A[N][N], float B[N][N], float C[N][N]) {
    int i = threadIdx.x;
    int j = threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}
```

Reflecting on Accessing Global Memory in CUDA

- Assume `dA` is a device array, holding `ints`
- Assume `z []` is an array of `unsigned ints`
- Which of these global memory accesses are good/bad? Why?
 1. `dA[threadIdx.x]`
 2. `dA[threadIdx.x+3]`
 3. `dA[7*threadIdx.x+3]`
 4. `dA[z[threadIdx.x]]`
- NOTE: When answering 1-4 above, imagine you're a thread and try to understand what the other 31 threads in the warp do when the warp executes the memory instruction